

Quantifying SZZ-Induced Label Noise in Just-In-Time Software Defect Prediction under Verification-Latency-Aware Online Evaluation

Puneet Deshwani

2025MCS013

Under the supervision of Dr. Santosh Singh Rathore

Department of Computer Science and Engineering
Atal Bihari Vajpayee Indian Institute of Information Technology and Management, Gwalior

October 2026

Contents

1. Overview
2. Motivation
3. Related Works
4. Research Gaps
5. Research Questions and Objectives
6. Dataset
7. Methodology
8. Results
9. Conclusion and Future Work
10. Timeline
11. References

Overview

- Just-in-time software defect prediction (JIT-SDP) predicts, at commit time, whether a code change is likely to introduce a defect, so that review and testing effort can be directed to risky changes [6].
- The models are trained on past commits. Each commit is described by change metrics and is labelled as clean or defect-introducing.
- These labels are not recorded in the repository. They are inferred by the SZZ algorithm, which traces the lines changed in a bug-fixing commit back to the commits that last modified them [11].
- Bug-fixing commits often contain unrelated changes (tangled commits), so SZZ produces wrong labels. Only 17% to 32% of the changed lines in bug-fixing commits contribute to the fix [5].

Overview

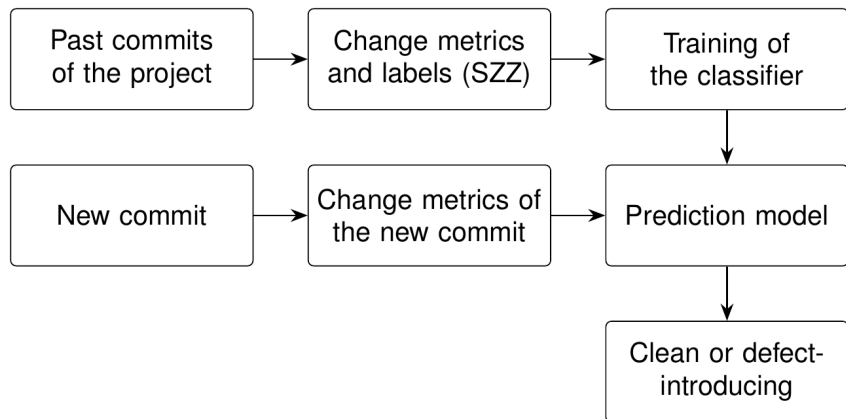


Figure 1: Workflow of JIT-SDP

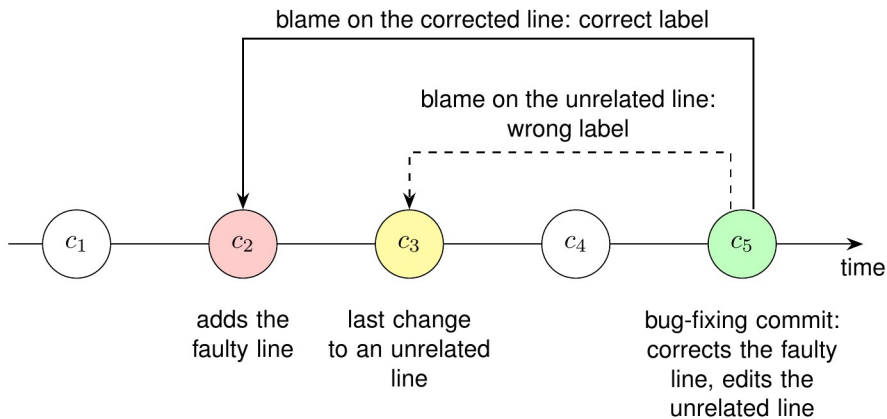


Figure 2: Labelling with SZZ and a wrong label caused by an unrelated change

Motivation

- JIT-SDP models are trained and tested on the same SZZ labels. A model is then rewarded for reproducing the errors of SZZ, and the effect of label noise on the reported performance is not visible.
- Random k-fold cross-validation is still widely used, although it trains the model on commits that are newer than the test commits [3].
- In practice the label of a commit arrives late (verification latency). In our data the median delay is 113 days, and 53% of the defect labels arrive after 90 days [1].
- It is therefore not known how much SZZ label noise costs a model that is evaluated in a realistic setting.

Related Works - 1

Title	Author	Dataset	Key Takeaways
When Do Changes Induce Fixes? (2005) [11]	Jacek Śliwerski	Mozilla and Eclipse	Proposes the SZZ algorithm: lines changed by a bug fix are traced back with annotate (blame) to find the bug-introducing changes
A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes (2017) [2]	Daniel A. da Costa	Ten open-source projects	Criteria for evaluating SZZ implementations; proposes MA-SZZ, which ignores meta-changes
Evaluating SZZ Implementations Through a Developer-Informed Oracle (2021) [10]	Giovanni Rosa	1,930 bug-fixing commits in which the developer names the introducing commit	Compares SZZ variants against the oracle; R-SZZ performs best (precision about 66%); releases PySZZ
A Fine-grained Data Set and Analysis of Tangling in Bug Fixing Commits (2022) [5]	Steffen Herbold	LLTC4J: manually labelled bug-fixing commits of Apache Java projects	Only 17% to 32% of the changed lines contribute to the bug fix; tangling adds noise to defect data
The Impact of Changes Mislabelled by SZZ on Just-in-Time Defect Prediction (2021) [4]	Yuanrui Fan	126,526 changes from ten Apache projects	Compares four SZZ variants with RA-SZZ as the reference; labels of AG-SZZ reduce performance significantly

Related Works - 2

Title	Author	Dataset	Key Takeaways
A Large-Scale Empirical Study of Just-in-Time Quality Assurance (2013) [6]	Yasutaka Kamei	Six open-source and five commercial projects	14 change metrics in five groups; 68% accuracy and 64% recall with logistic regression
Are Fix-Inducing Changes a Moving Target? (2018) [7]	Shane McIntosh	Qt and OpenStack (37,524 changes)	Properties of fix-inducing changes change over time; models lose predictive power as they age
Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction (2019) [1]	George G. Cabral	Ten GitHub projects	Online JIT-SDP with delayed labels (90-day waiting period); proposes ORB for evolving class imbalance
Deep Just-in-Time Defect Prediction: How Far Are We? (2021) [12]	Zhengran Zeng	Open-source projects with over 310,000 changes	Deep models do not consistently beat simple ones; LAPredict (logistic regression on added lines) is best in most cases
The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization (2022) [8]	Chao Ni	JIT-Defects4J: 21 Java projects, 27,319 commits	Dataset built from manually validated bug-fixing lines; proposes JIT-Fine

Research Gaps

- Noise of SZZ labels not measured over complete project histories
- Models tested against the same labels on which they are trained
- Effect of label noise not separated from the effect of the evaluation procedure
- Label noise not studied under verification latency
- Effect of SZZ noise on online learners not understood, and no remedy for this setting

Research Questions

- **RQ1:** How much do the labels of SZZ variants disagree with one another and with independent reference labels, and what type of error does each variant make?
- **RQ2:** How does the choice of the label source affect the measured performance of JIT-SDP models under different evaluation settings?
- **RQ3:** Through which mechanism does label noise reduce the performance of an online learner, and which type of error is responsible? (planned for MTP III)
- **RQ4:** Can an online learner be made less sensitive to SZZ label noise without access to clean labels? (planned for MTP IV)

Objectives

- **Objective 1:** Generate the labels of six SZZ variants for a dataset with independent reference labels, and characterise their false positive and false negative rates.
- **Objective 2:** Examine the agreement between the variants and identify the sources of their errors.
- **Objective 3:** Measure how much the evaluation procedure changes the measured performance of JIT-SDP models.
- **Objective 4:** Measure the loss caused by SZZ labels for an online learner under verification latency.
- **Objective 5:** Identify the type of labelling error responsible for the loss and its mechanism. (MTP III)
- **Objective 6:** Design and evaluate a noise-aware online learner. (MTP IV)

Dataset

- JIT-Defects4J [8]: 27,319 commits from 21 Apache Java projects (2001 to 2019)
- 2,332 commits (8.54%) are defect-introducing;
5,453 commits are bug-fixing
- Reference labels: bug-fixing lines validated by at least three of four annotators, then traced back with git blame
- Features: the 14 change metrics of Kamei et al. [6]
- Seven labels per commit: reference label and six SZZ variants
- Label arrival times taken from the dates of the bug-fixing commits (median delay 113 days)

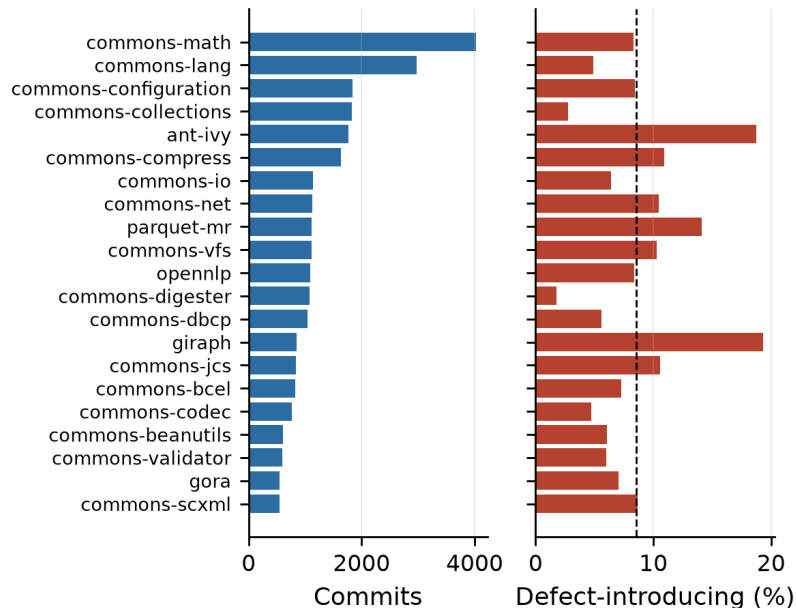


Figure 3: Commits and defect rate of each project

Methodology

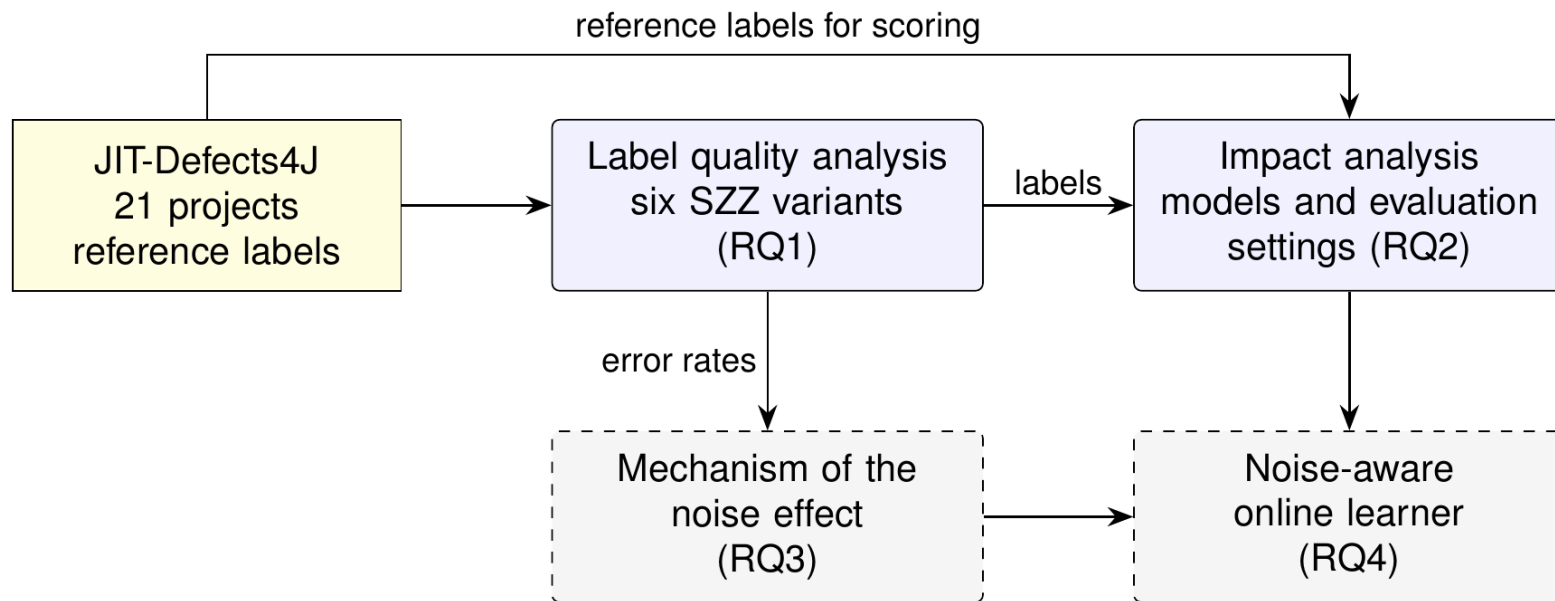


Figure 4: Overview of the work. Solid boxes: MTP II (completed). Dashed boxes: MTP III and MTP IV (planned)

Methodology

- **SZZ labels:** six variants (B-, AG-, MA-, L-, R- and RA-SZZ) run with PySZZ on the same 5,453 bug-fixing commits
- **Label quality:** precision, recall, false-alarm rate p_0 , miss rate p_1 and Cohen's κ against the reference labels
- **Models:** LApredict (logistic regression on lines added) [12], a JITLine-based random forest with SMOTE [9], and ORB (online ensemble) [1]
- **Scoring:** against the held-out reference labels, and against the training labels (self-scoring) for comparison
- **Statistics:** paired by project ($n = 21$), Wilcoxon signed-rank test, Hodges-Lehmann estimate with bootstrap confidence interval, Holm correction over all 74 tests
- **Scale:** 3 models, 7 label sources, 4 evaluation settings, 10 seeds, 16,380 runs

Methodology: Evaluation Settings

(a) Random k -fold cross-validation (one fold shown)



(b) Chronological split and chronological-online



(c) Prequential evaluation with verification latency

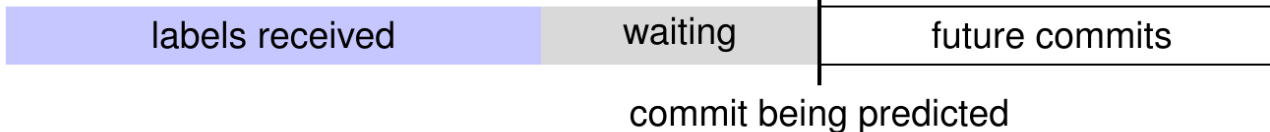


Figure 5: Evaluation settings. Commits of a project are ordered by time from left to right

Results: Quality of SZZ Labels (RQ1)

Variant	Flagged	Precision	Recall	ρ_0	ρ_1	κ
B-SZZ	8,060	0.186	0.641	0.263	0.359	0.179
AG-SZZ	5,876	0.186	0.467	0.192	0.533	0.163
MA-SZZ	6,154	0.183	0.482	0.201	0.518	0.161
L-SZZ	2,287	0.272	0.267	0.067	0.733	0.202
R-SZZ	3,015	0.232	0.300	0.093	0.700	0.183
RA-SZZ	5,553	0.184	0.438	0.181	0.562	0.158

- Precision is at most 0.272: more than 72% of the flagged commits are clean according to the reference
- The noise is class-conditional: B-SZZ mainly gives false positives, L-SZZ and R-SZZ mainly miss defects

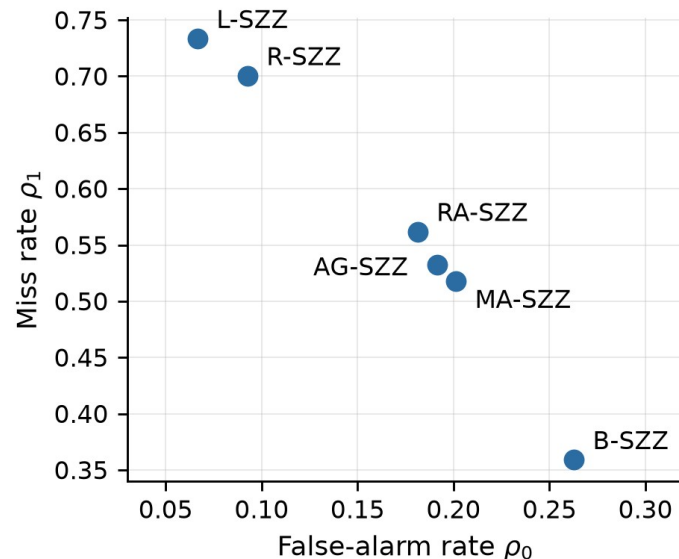


Figure 6: False-alarm rate and miss rate

Results: Agreement between Variants (RQ1)

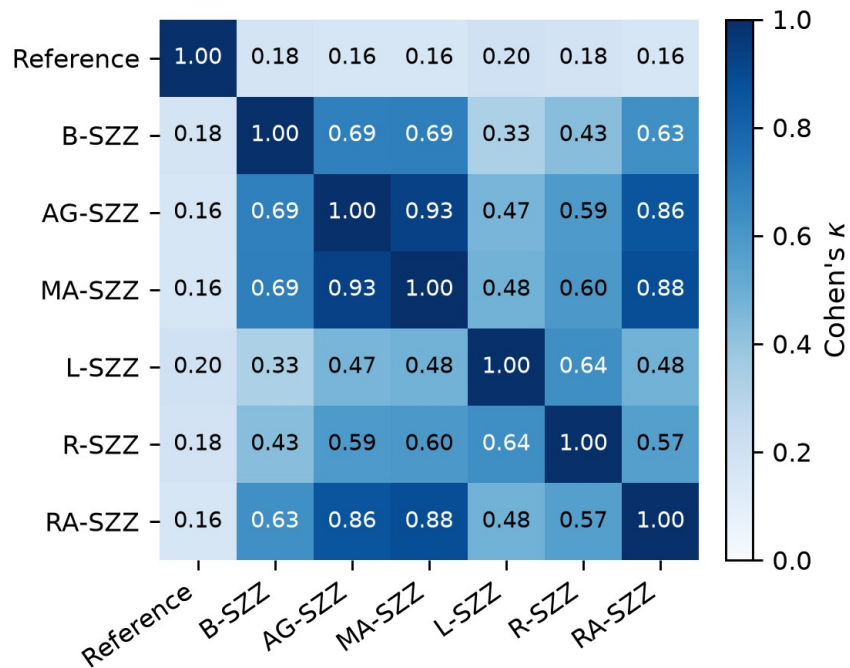


Figure 7: Cohen's κ between the label sets

- Agreement with the reference labels: $\kappa = 0.16$ to 0.20
- Agreement among the variants: $\kappa = 0.33$ to 0.93
- Agreement between SZZ variants is therefore not evidence of correct labels
- Filtering also removes correct labels: 30% to 60% of the true positives of B-SZZ are lost in the refined variants

Results: Effect of the Evaluation Setting (RQ2)

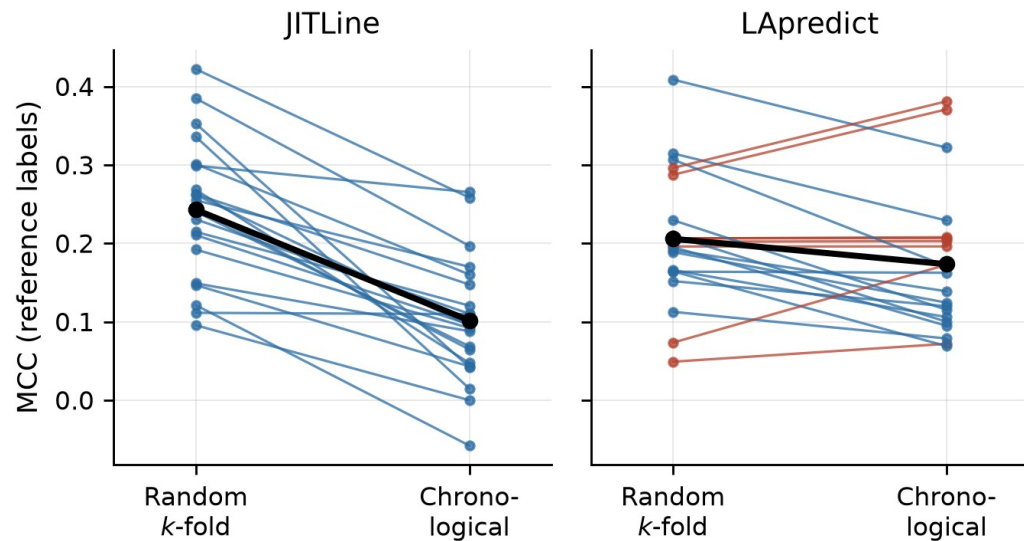


Figure 8: MCC of each project (models trained on reference labels)

- Random k-fold gives a higher MCC than the chronological split in all 14 comparisons
- JITLine: 0.243 against 0.102 (+0.137), in 21 of 21 projects
- LAPredict: +0.034, not significant
- With the chronological split the simpler model is better (0.173 against 0.102)

Results: Effect of Scoring against Training Labels (RQ2)

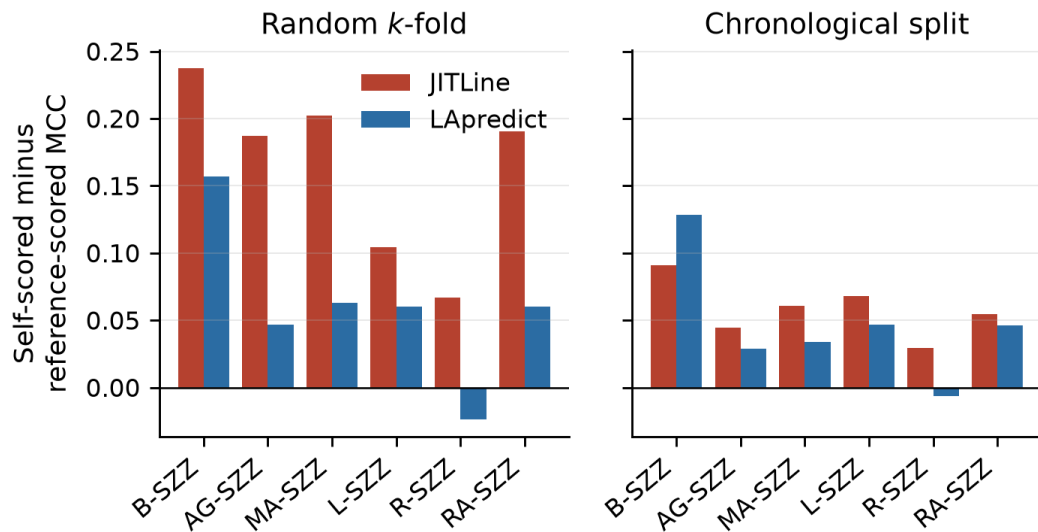


Figure 9: Self-scored minus reference-scored MCC

- Scoring a model against its own SZZ labels gives a higher MCC than scoring against the reference
- JITLine with B-SZZ (k-fold): 0.413 against 0.175 (+0.230)
- Significant for five of six variants with JITLine, only for B-SZZ with LAPredict
- The errors of B-SZZ are learnable: AUC 0.60 against 0.51 with permuted labels

Results: Effect of the Label Source, Online Setting (RQ2)

Training labels	MCC	Loss	95% CI	Projects	p (corrected)
Reference	0.097	-	-	-	-
B-SZZ	0.058	0.041	[0.022, 0.052]	18/21	0.006
AG-SZZ	0.036	0.060	[0.029, 0.085]	17/21	0.024
MA-SZZ	0.035	0.064	[0.039, 0.085]	18/21	0.004
L-SZZ	0.058	0.034	[0.014, 0.055]	16/21	0.124
R-SZZ	0.032	0.060	[0.042, 0.082]	19/21	< 0.001
RA-SZZ	0.039	0.058	[0.036, 0.081]	18/21	0.006

- ORB evaluated on the commit stream with verification latency, scored against the reference labels
- The model trained on reference labels is better than all six SZZ-trained models
- Loss of 0.034 to 0.064 MCC, significant for five of six variants after correction over all tests

Results: Combined View

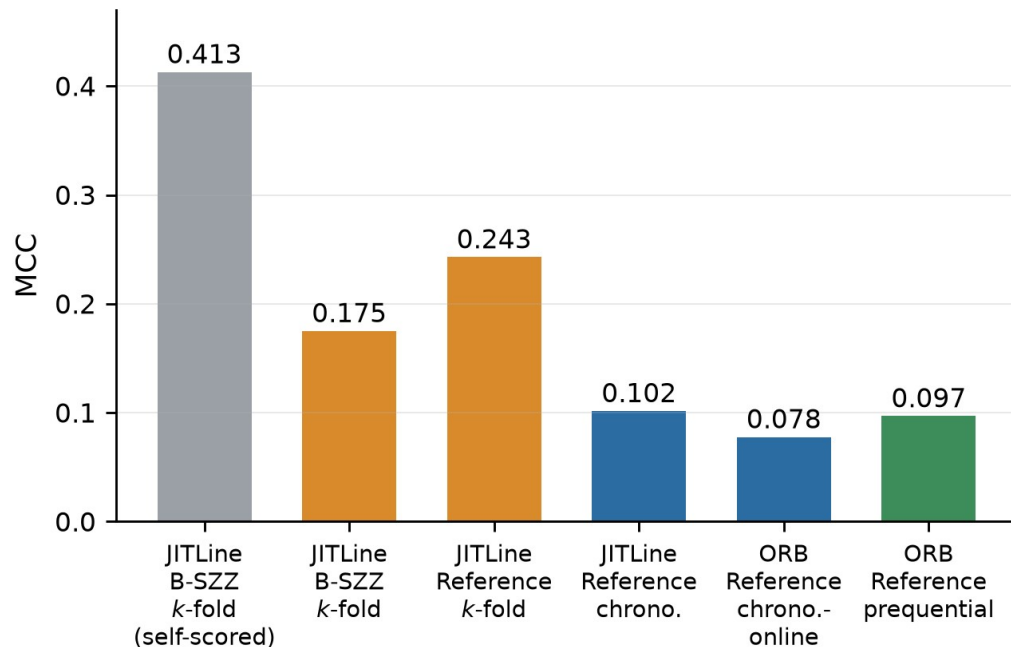


Figure 10: MCC under increasingly realistic conditions

- Common practice (JITLine, B-SZZ labels, k-fold, self-scored): 0.413
- Same predictions scored against reference labels: 0.175
- Reference labels with chronological split: 0.102
- Online learner with verification latency: 0.097
- The evaluation procedure accounts for more of the difference than the label noise

Conclusion

- The labels of all six SZZ variants differ strongly from the reference labels (precision 0.18 to 0.27). The noise is class-conditional and depends on the variant.
- SZZ variants agree with each other much more than with the reference, so their agreement does not indicate correct labels.
- Random k-fold cross-validation and scoring against the training labels overestimate the performance (+0.137 and up to +0.230 MCC), more so for the complex model.
- In the online setting with verification latency, training on SZZ labels loses 0.034 to 0.064 MCC compared with the reference labels (0.097).
- RQ1 and RQ2 are answered in MTP II. The cause of the loss and a remedy are the subject of MTP III and MTP IV.

Future Work

- **MTP III:** identify the cause of the loss (RQ3) by injecting noise into the reference labels at the measured rates, and by repairing the B-SZZ labels one error type at a time
- **Controls in MTP III:** equal number of corrected labels of each type, immediate delivery of repaired labels, and a condition without label delay; the resampling rate of ORB will be recorded
- **MTP IV:** design a noise-aware online learner (RQ4) that improves the performance on SZZ labels without reducing it on clean labels, with the evaluation plan fixed before the experiments
- **Further evaluation:** effort-aware measures and other online learners, if time permits

Timeline

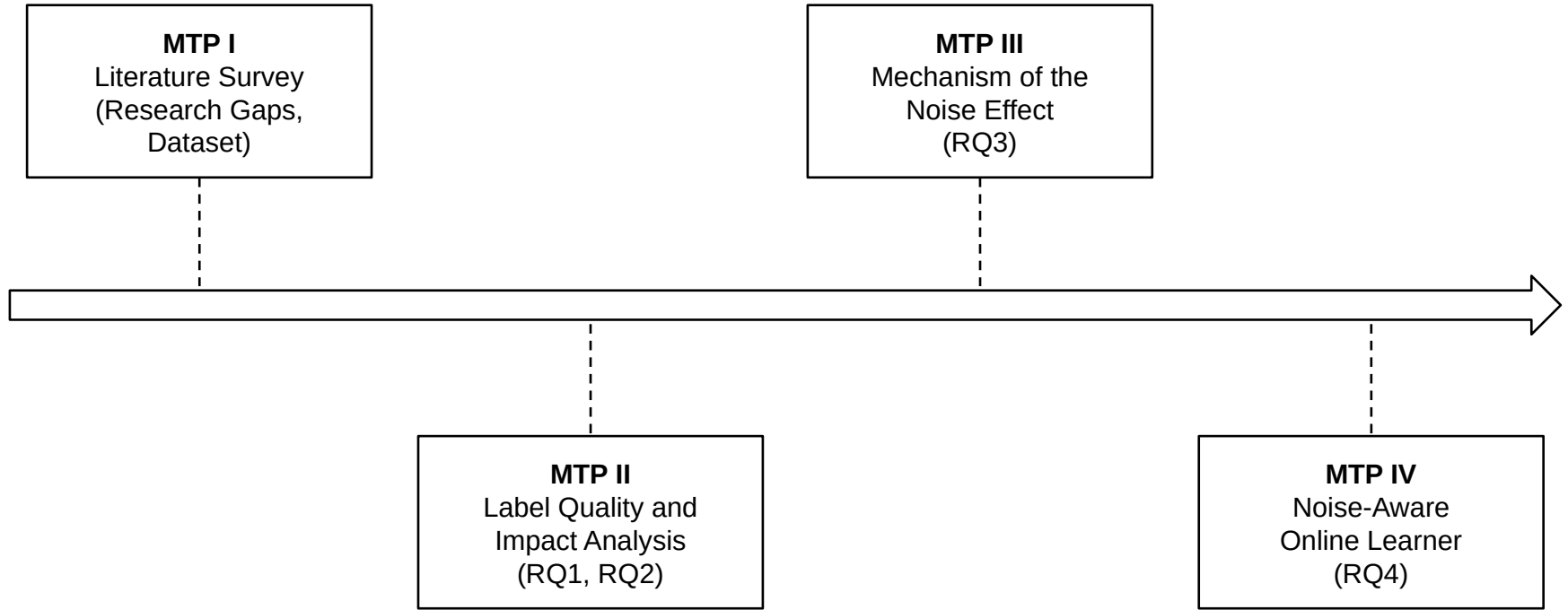


Figure 11: Timeline

References I

- [1] George G. Cabral et al. “Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction”. In: *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 666-676.
- [2] Daniel Alencar da Costa et al. “A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes”. In: *IEEE Transactions on Software Engineering* 43.7 (2017), pp. 641-657.
- [3] Davide Falessi et al. “On the Need of Preserving Order of Data When Validating Within-Project Defect Classifiers”. In: *Empirical Software Engineering* 25 (2020), pp. 4805-4830.
- [4] Yuanrui Fan et al. “The Impact of Changes Misabeled by SZZ on Just-in-Time Defect Prediction”. In: *IEEE Transactions on Software Engineering* 47.8 (2021), pp. 1559-1586.
- [5] Steffen Herbold et al. “A Fine-grained Data Set and Analysis of Tangling in Bug Fixing Commits”. In: *Empirical Software Engineering* 27.6 (2022), p. 125.
- [6] Yasutaka Kamei et al. “A Large-Scale Empirical Study of Just-in-Time Quality Assurance”. In: *IEEE Transactions on Software Engineering* 39.6 (2013), pp. 757-773.
- [7] Shane McIntosh and Yasutaka Kamei. “Are Fix-Inducing Changes a Moving Target? A Longitudinal Case Study of Just-in-Time Defect Prediction”. In: *IEEE Transactions on Software Engineering* 44.5 (2018), pp. 412-428.
- [8] Chao Ni et al. “The Best of Both Worlds: Integrating Semantic Features with Expert Features for Defect Prediction and Localization”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 2022, pp. 672-683.

References II

- [9] Chanathip Pornprasit and Chakkrit Tantithamthavorn. “JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction”. In: *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*. 2021, pp. 369-379.
- [10] Giovanni Rosa et al. “Evaluating SZZ Implementations Through a Developer-Informed Oracle”. In: *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. 2021.
- [11] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. “When Do Changes Induce Fixes?”. In: *Proceedings of the International Workshop on Mining Software Repositories (MSR)*. 2005, pp. 1-5.
- [12] Zhengran Zeng et al. “Deep Just-in-Time Defect Prediction: How Far Are We?”. In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2021.